

CSC 4227 Digital Image Processing

Lecture 07 – Spatial Filtering

Dr. Kazi A Kalpoma

Professor & Head

Computer Science and Engineering

Ahsanullah University of Science & Technology (AUST)

Contact: kalpoma@gmail.com

Today's Contents

- ❑ Neighborhood operations
- ❑ What is spatial filtering?
- ❑ Why filters are used?
- ❑ Types of filters
- ❑ Smoothing operations
- ❑ Correlation and convolution

■ Chapter 3 from R.C. Gonzalez and R.E. Woods, Digital Image Processing (3rd Edition), Prentice Hall, 2008 [**Section 3.4, 3.5**]

Spatial Filtering

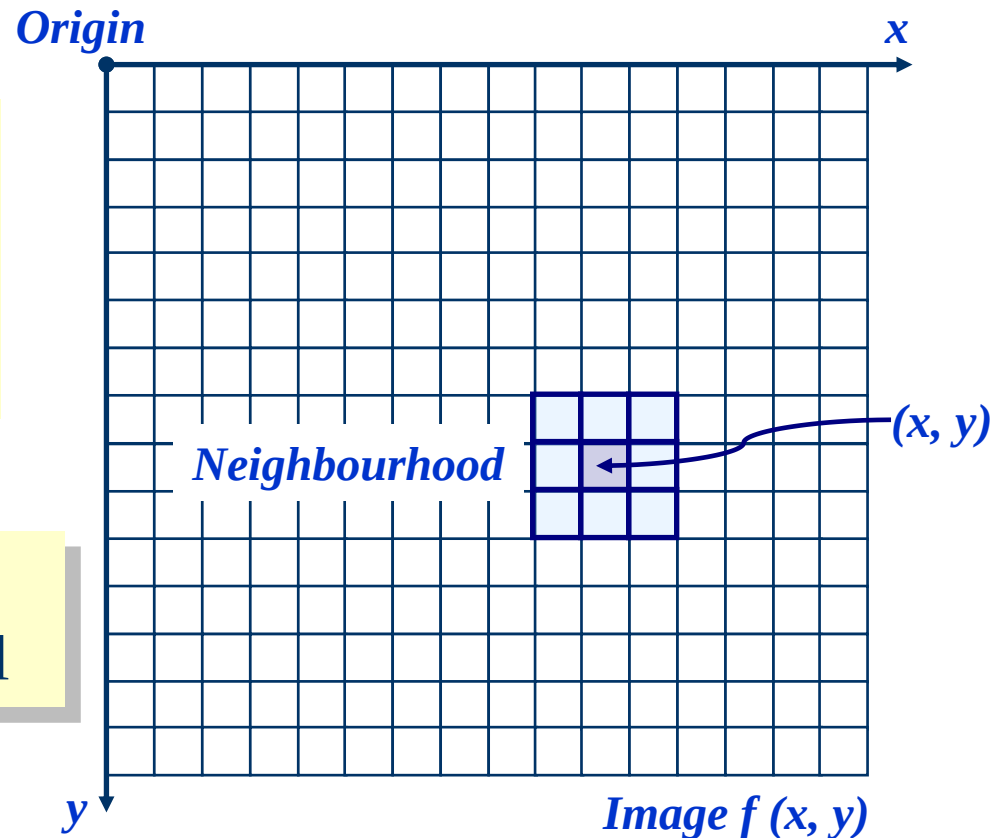
- ❑ A spatial filter consists of
 - (i) a **neighborhood**, and
 - (ii) a **predefined operation** (filter/mask/kernel)
- ❑ Linear spatial filtering of an image of size MxN with a filter of size mnxn is given by the expression

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Neighborhood operations

Neighbourhood operations:
Operate on a larger
neighbourhood of pixels
than point operations

Neighbourhoods are mostly a
rectangle around a central pixel



Simple Neighbourhood Operations

- **Min:** Set the pixel value to the minimum in the neighbourhood
- **Max:** Set the pixel value to the maximum in the neighbourhood
- **Median:** The median value of a set of numbers is the midpoint value in that set (e.g. from the set [1, 7, 15, 17, 18, 21, 22, 23, 24] **18 is the median**). Sometimes the median works better than the average

Simple Neighbourhood Operations

Example

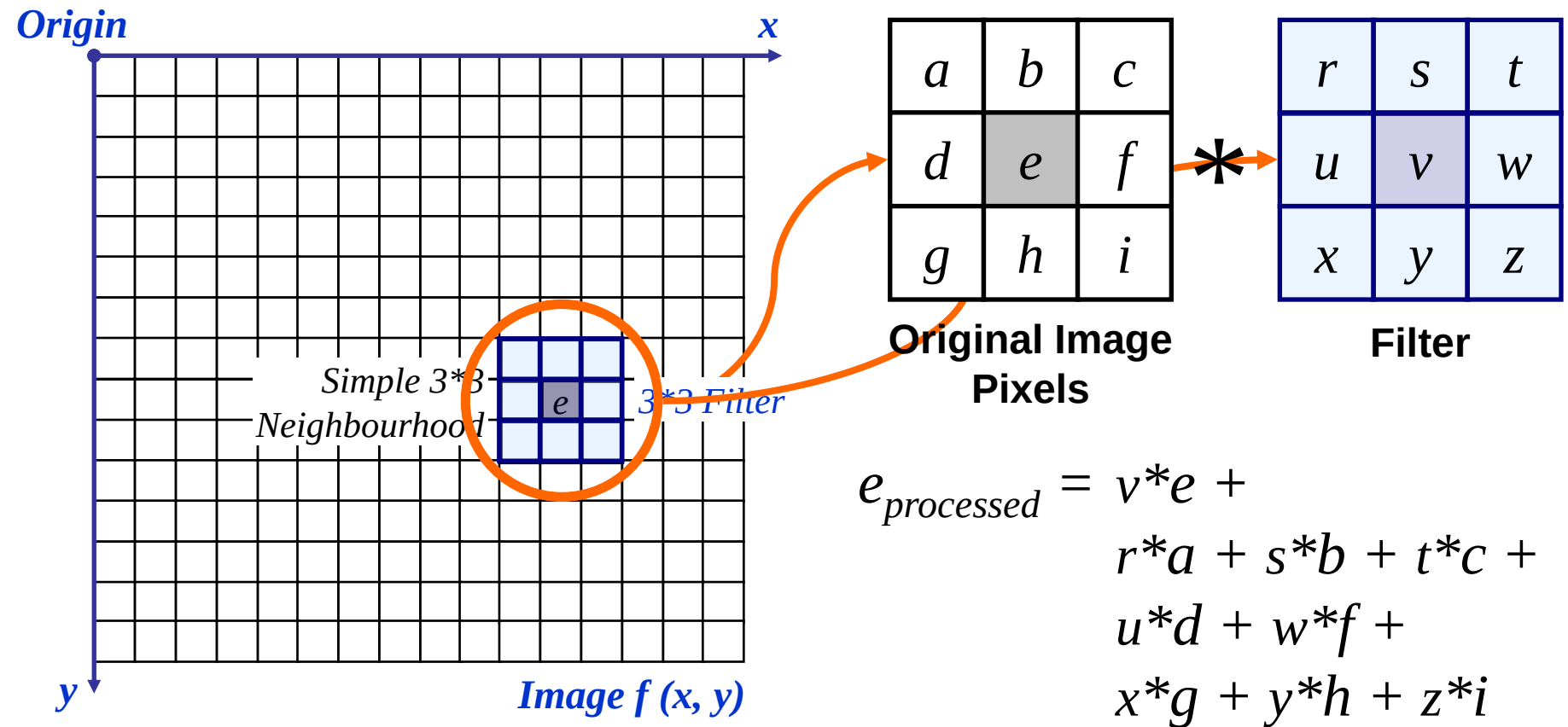
Original Image

123	127	128	119	115	130	...
140	145	148	153	167	172	
133	154	183	192	194	191	
194	199	207	210	198	195	
164	170	175	162	173	151	
...						

Enhanced Image

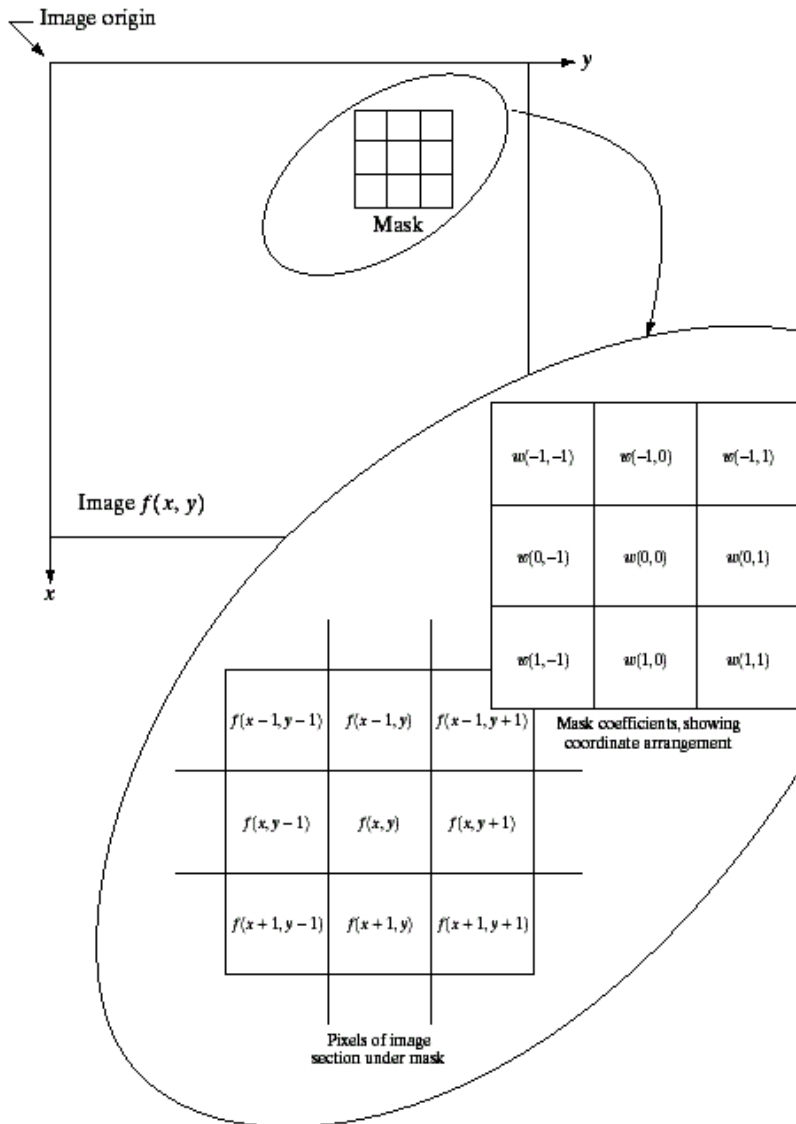
						...
...						

The Spatial Filtering Process



The above is repeated for every pixel in the original image to generate the filtered image

Spatial Filtering: Equation Form



$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

where $a = \frac{m-1}{2}$, $b = \frac{n-1}{2}$

$x = 0, 1, 2, \dots, M-1$, $y = 0, 1, 2, \dots, N-1$

Filtering can be given in equation form as shown above

$m \times n$ is the size of mask

Why filters are used?

- ❑ Filters are applied on image for multiple purposes.
- ❑ Most common uses are as following:
 - ❑ Filters are used for **Blurring** and **noise** reduction
 - ❑ Filters are used for **edge detection** and **sharpness**
- ❑ Really important!
 - **Enhance images**
 - De-noise, resize, increase contrast, etc.
 - **Extract information from images**
 - Texture, edges, distinctive points, etc.
 - **Detect patterns**
 - Template matching

Smoothing Spatial Filters:

Blurring and Noise reduction

❑ The common masks for blurring are.

- Mean filter or Box filter

- Weighted average filter

❑ In the process of blurring we reduce the edge content in an image and try to make the transitions between different pixel intensities as smooth as possible.

❑ Noise reduction is also possible with the help of blurring.

Mean filter

□ Mean filter is also known as Box filter and Average filter. A mean filter has the following **properties**.

➤ It must be **odd** ordered.

➤ The sum of all the elements should be **1**.

➤ All the elements should be **same**.

➤ If we follow this rule, then for a mask of 3x3. We get the following result.

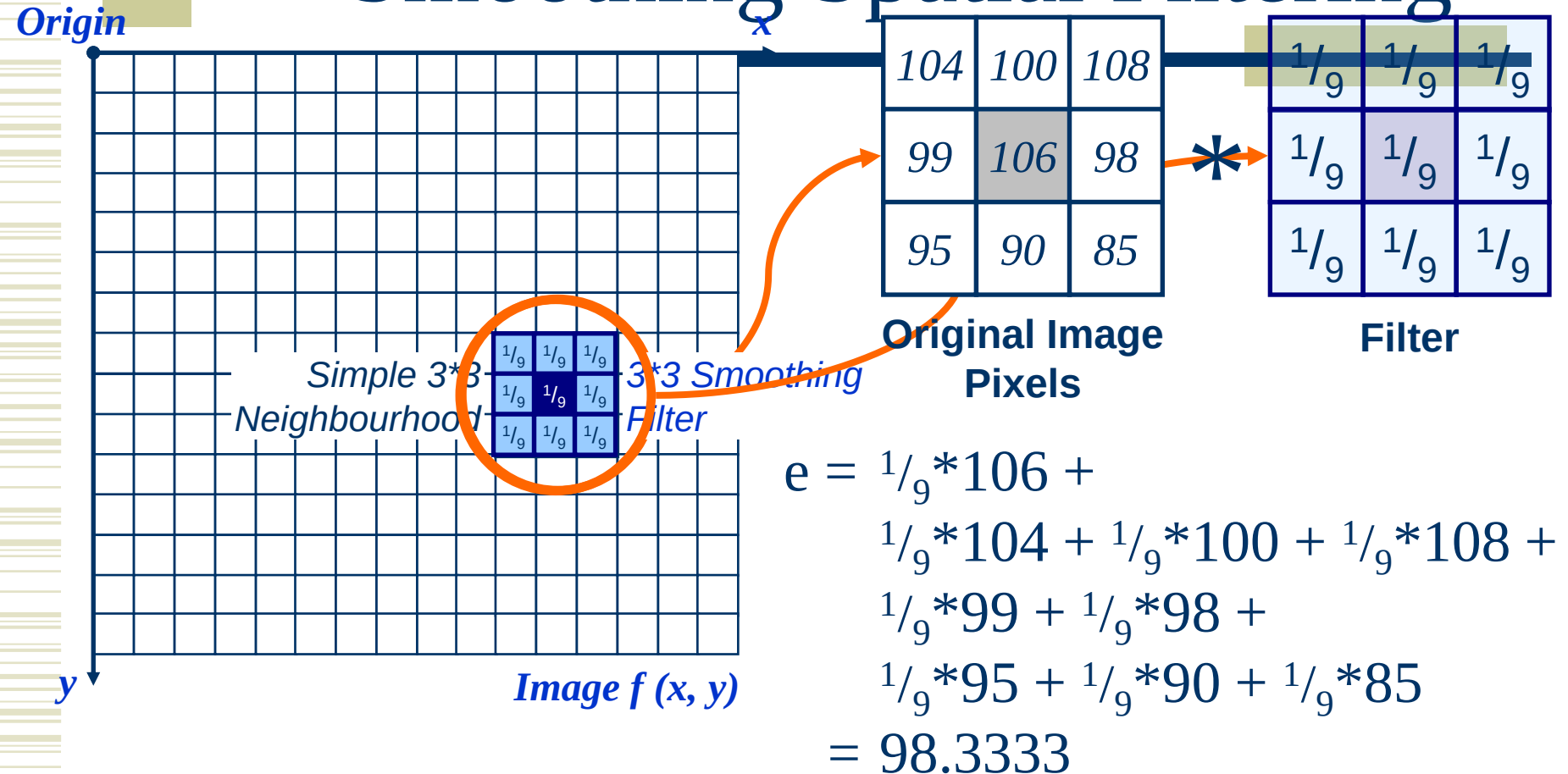
$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1

Since it is a 3x3 mask, that means it has 9 cells. The condition that all the element sum should be equal to 1 can be achieved by dividing each value by 9. As

$$1/9 + 1/9 + 1/9 + 1/9 + 1/9 + 1/9 + 1/9 + 1/9 + 1/9 = 9/9 = 1$$

Smoothing Spatial Filtering



The above is repeated for every pixel in the original image to generate the smoothed image

Image filtering in 2D (Mean Filter)

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	90	0	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20						

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				
						?			
				50					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

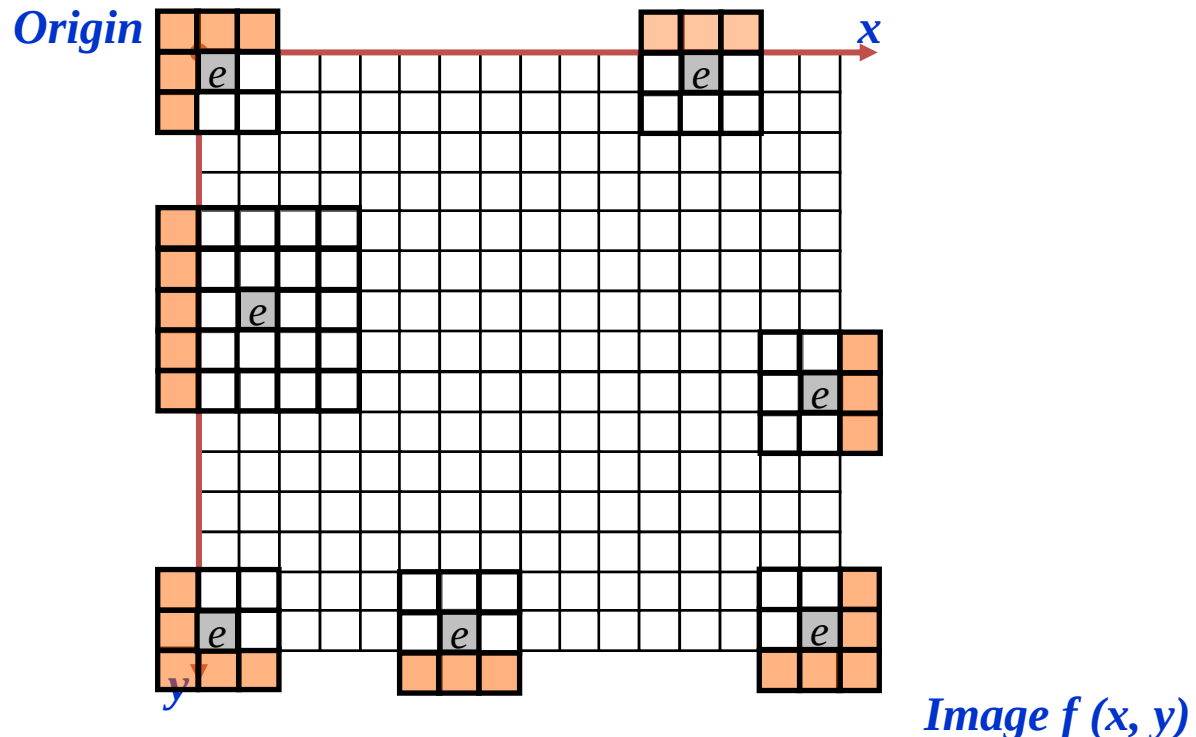
$h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Strange Things Happen At The Edges!

At the edges of an image there are missing pixels which form a neighbourhood



Mask operation near the image border: Problem arises when part of the mask is located outside the image plane

Discard the problem pixels (e.g. 512x512 input 510x510 output if mask size is 3x3)

Zero padding: Expand the input image by padding zeros (512x512 original image, 514x514 padded image, 512x512 output)

Zero padding is not recommended as it creates artificial lines or edges on the border

Pixel replication: We normally use the gray levels of border pixels to fill up the expanded region (for 3x3 mask). For larger masks a border region equal to half of the mask size is mirrored on the expanded region.

What does Mean or Box filter do?

kernel for a 3x3 mean filter

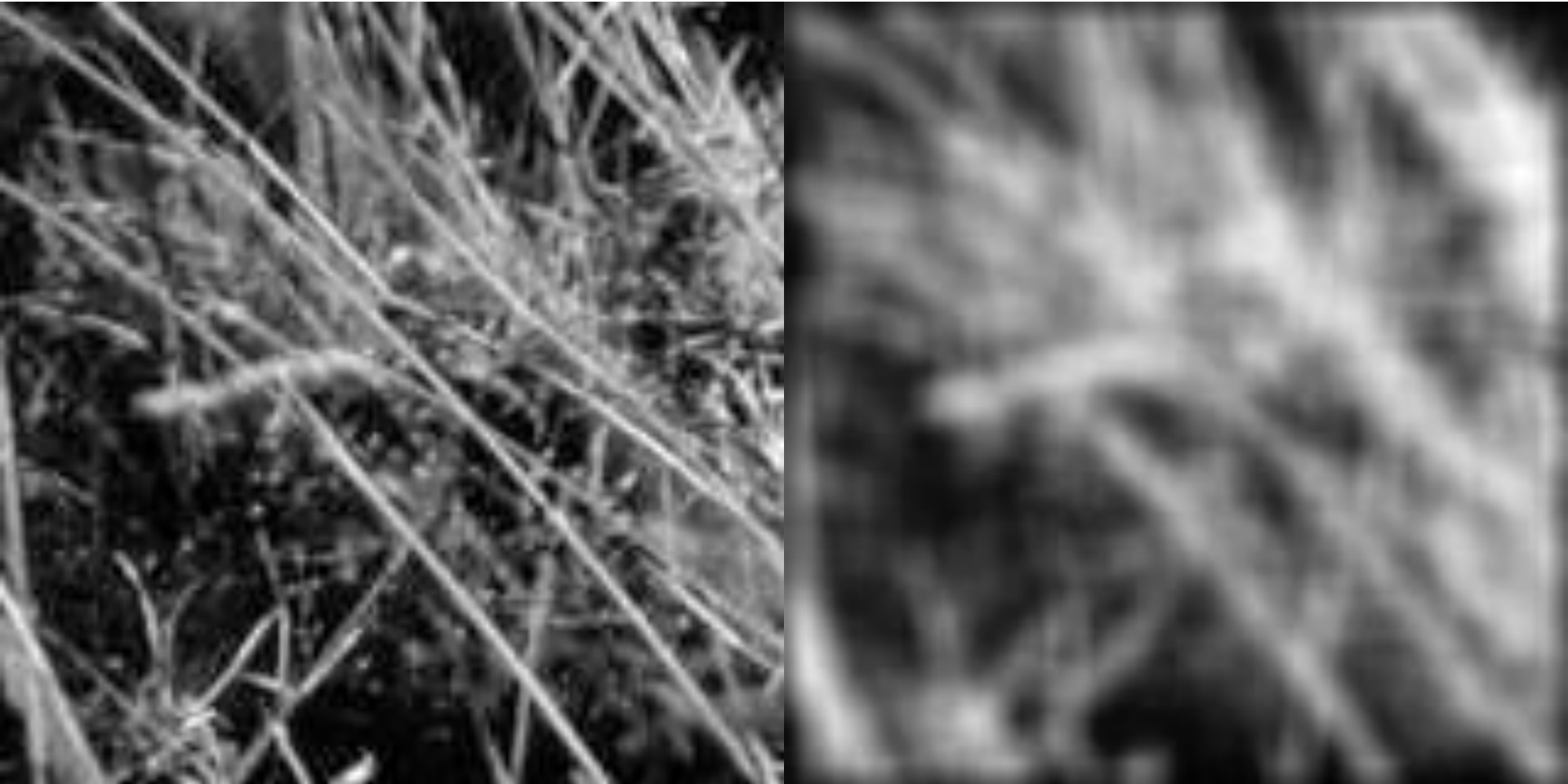
$g[\cdot, \cdot]$

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect
(remove sharp features)
- Adds a “softer” look to an image

$$\frac{1}{9}$$

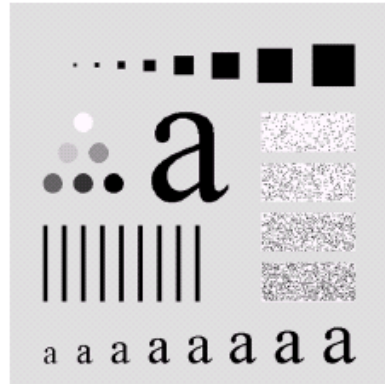
1	1	1
1	1	1
1	1	1

Smoothing with box filter

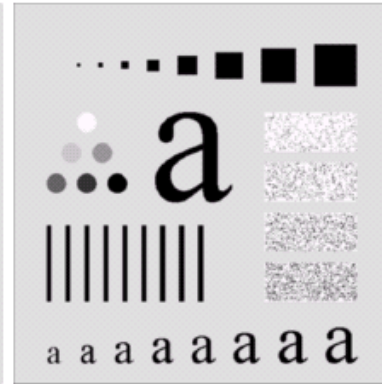


Original image

Size: 500x500



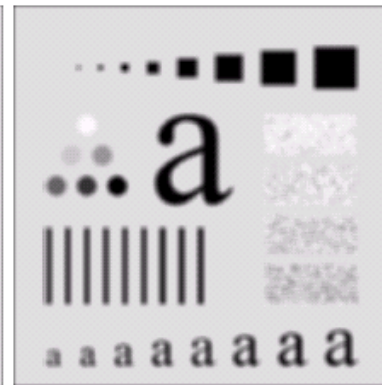
**Smooth by 3x3
box filter**



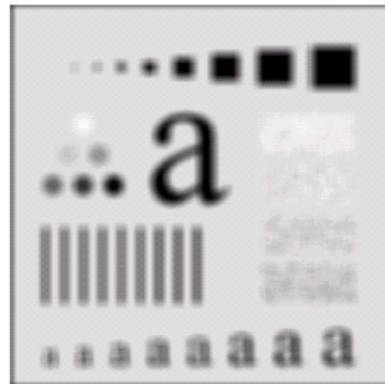
**Smooth by 5x5
box filter**



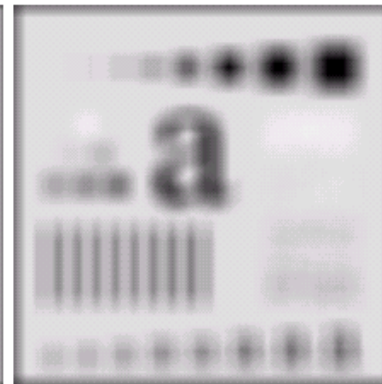
**Smooth by 9x9
box filter**



**Smooth by
15x15 box filter**



**Smooth by
35x35 box filter**



Notice how detail begins to disappear

Weighted Smoothing Filters

- More effective smoothing filters can be generated by allowing different pixels in the neighbourhood different weights in the averaging function

- Pixels closer to the central pixel are more important
- Often referred to as a *weighted averaging*

$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$
$\frac{2}{16}$	$\frac{4}{16}$	$\frac{2}{16}$
$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$

Weighted
averaging filter

Smoothing Spatial Filters

$\frac{1}{9} \times$

1	1	1
1	1	1
1	1	1

Box Filter all coefficients are equal

$\frac{1}{16} \times$

1	2	1
2	4	2
1	2	1

Weighted Average give more (less) weight to near (away from) the output location

Consider the output pixel is positioned at the center

Spatial Filtering:

with coefficient mask

- Given the 3×3 mask with coefficients: $w_1, w_2, \dots, w_9$
- The mask cover the pixels with gray levels: $z_1, z_2, \dots, z_9$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

Sum of Product

$$z \longleftarrow z_1 w_1 + z_2 w_2 + z_3 w_3 + \dots + z_9 w_9 = \sum_{i=1}^9 z_i w_i$$

- z gives the output intensity value for the processed image (to be stored in a new array) at the location of z_5 in the input image

Spatial Correlation & Convolution

❑ The filtering we have been talking about so far is referred to as *correlation*

❑ *Correlation* is the process of moving a filter mask/window/kernel over the image and computing the *sum of product*.

❑ *Convolution* is a similar correlation operation except the filter is flipped both horizontally and vertically before being applied to the image.

❑ For *symmetric filters* it makes no difference

Correlation

$$f \otimes h = \sum_{s=-a}^a \sum_{t=-b}^b h(s,t) f(x+s, y+t)$$

f = Image

h = Kernel

f

f_1	f_2	f_3
f_4	f_5	f_6
f_7	f_8	f_9

$\otimes$

h

h_1	h_2	h_3
h_4	h_5	h_6
h_7	h_8	h_9

Sum of Product

$$\begin{aligned} f * h &= f_1 h_1 + f_2 h_2 + f_3 h_3 \\ &+ f_4 h_4 + f_5 h_5 + f_6 h_6 \\ &+ f_7 h_7 + f_8 h_8 + f_9 h_9 \end{aligned}$$

Convolution

$$f * h = \sum_{s=-a}^a \sum_{t=-b}^b h(s, t) f(x-s, y-t)$$

f = Image

h = Kernel

h_7	h_8	h_9
h_4	h_5	h_6
h_1	h_2	h_3

$X - flip$

h_1	h_2	h_3
h_4	h_5	h_6
h_7	h_8	h_9

h

f

f_1	f_2	f_3
f_4	f_5	f_6
f_7	f_8	f_9

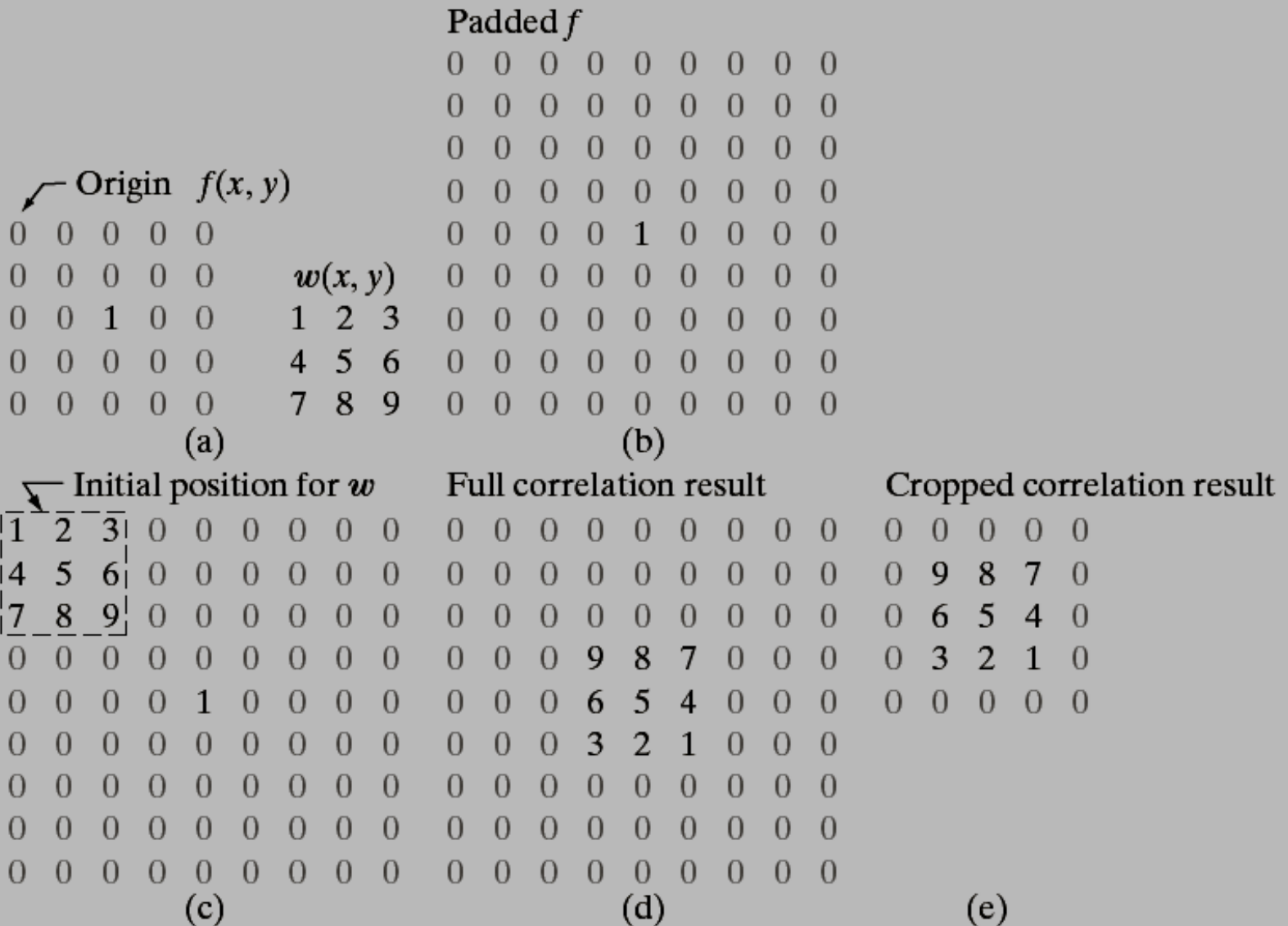
$\otimes$

$Y - flip$

h_9	h_8	h_7
h_6	h_5	h_4
h_3	h_2	h_1

$$\begin{aligned} f * h = & f_1 h_9 + f_2 h_8 + f_3 h_7 \\ & + f_4 h_6 + f_5 h_5 + f_6 h_4 \\ & + f_7 h_3 + f_8 h_2 + f_9 h_1 \end{aligned}$$

Example: 2D Spatial Correlation

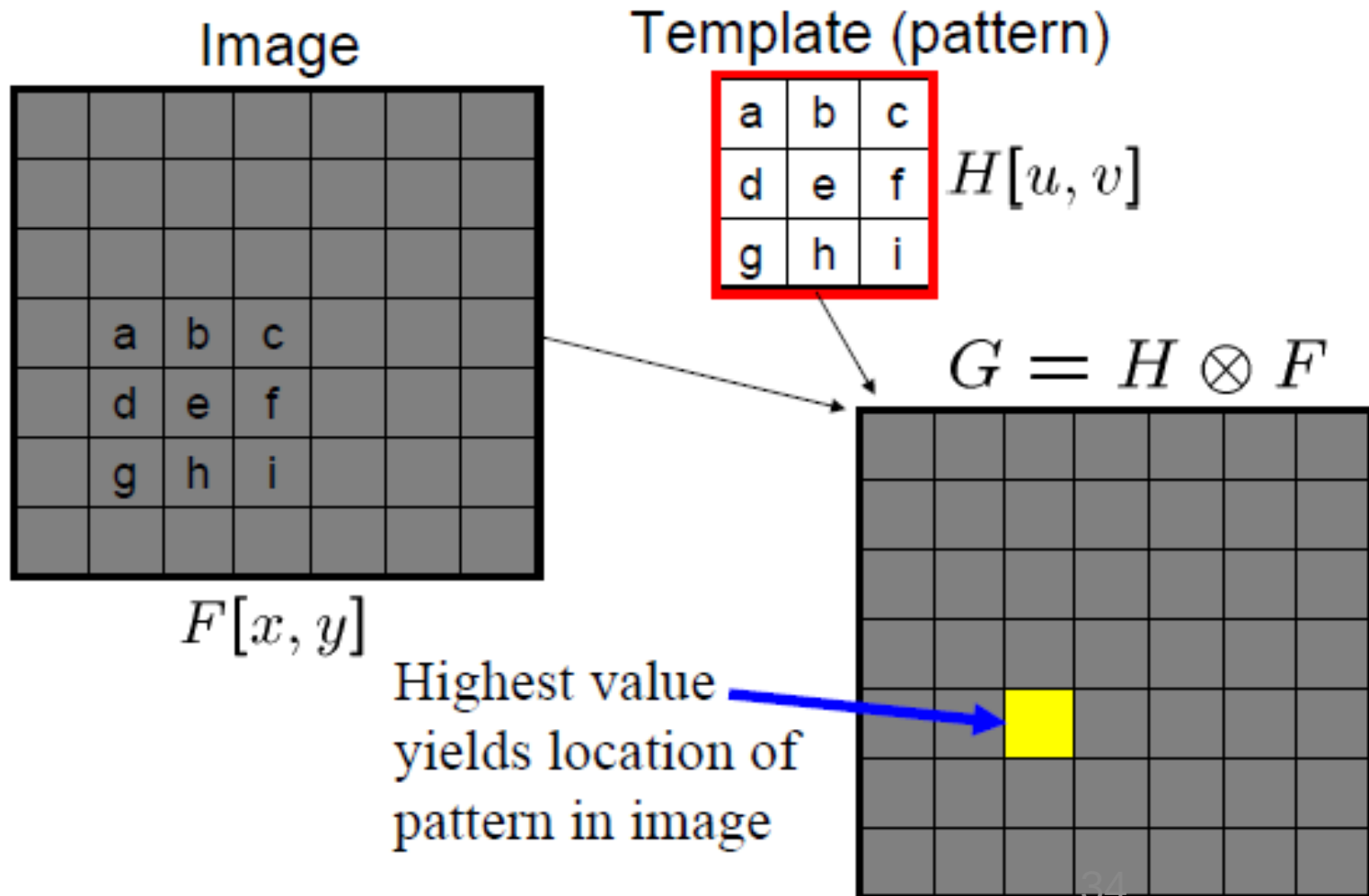


2D Spatial Convolution

Rotated w	Full convolution result	Cropped convolution result
9 8 7 6 5 4 3 2 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 2 3 0 0 0 0 0 0 4 5 6 0 0 0 0 0 0 7 8 9 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0 0 0 0 0 0 1 2 3 0 0 4 5 6 0 0 7 8 9 0 0 0 0 0 0
(f)	(g)	(h)

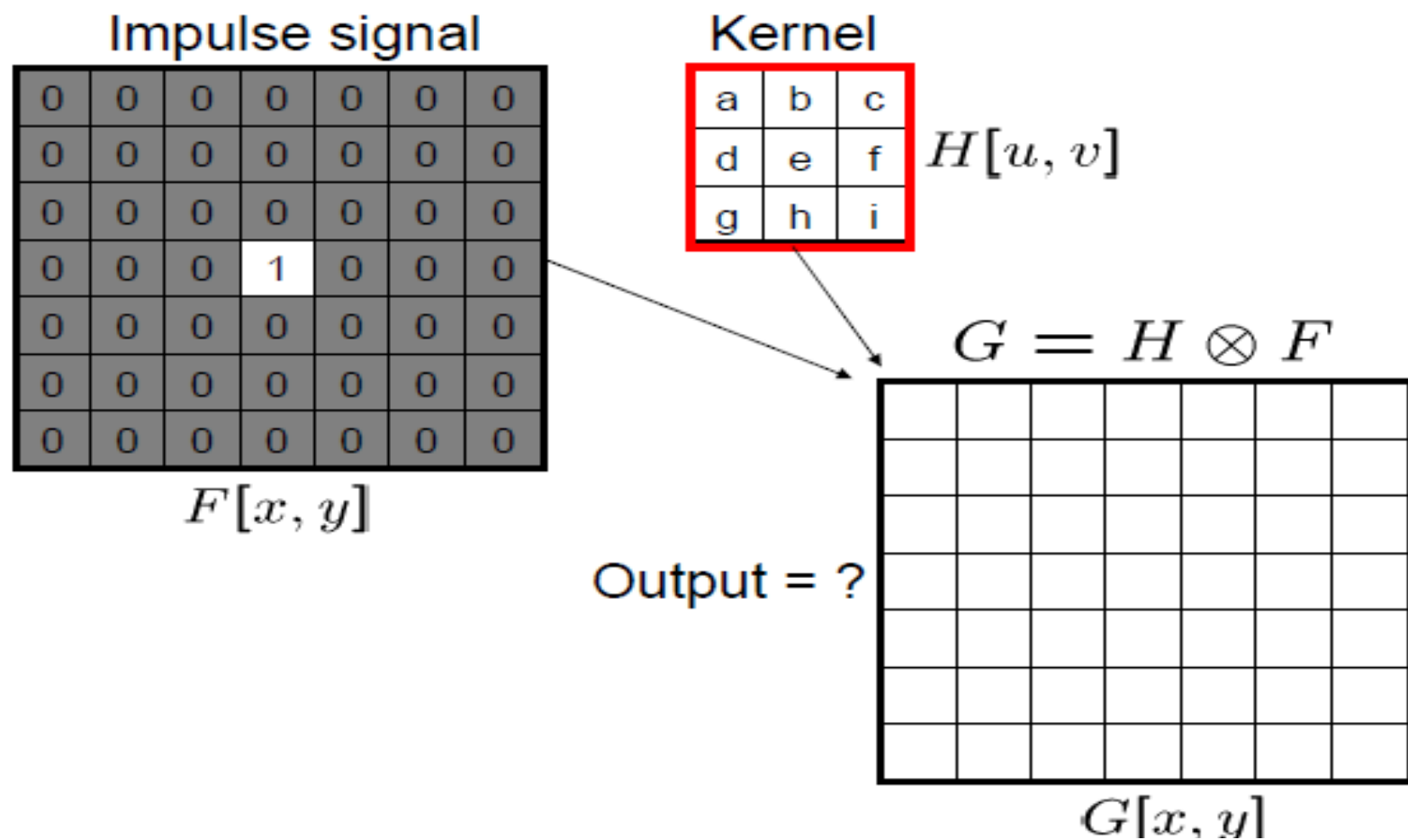
Cross-correlation and template matching

Cross-correlation is useful for *template matching* (locating a given pattern in an image)

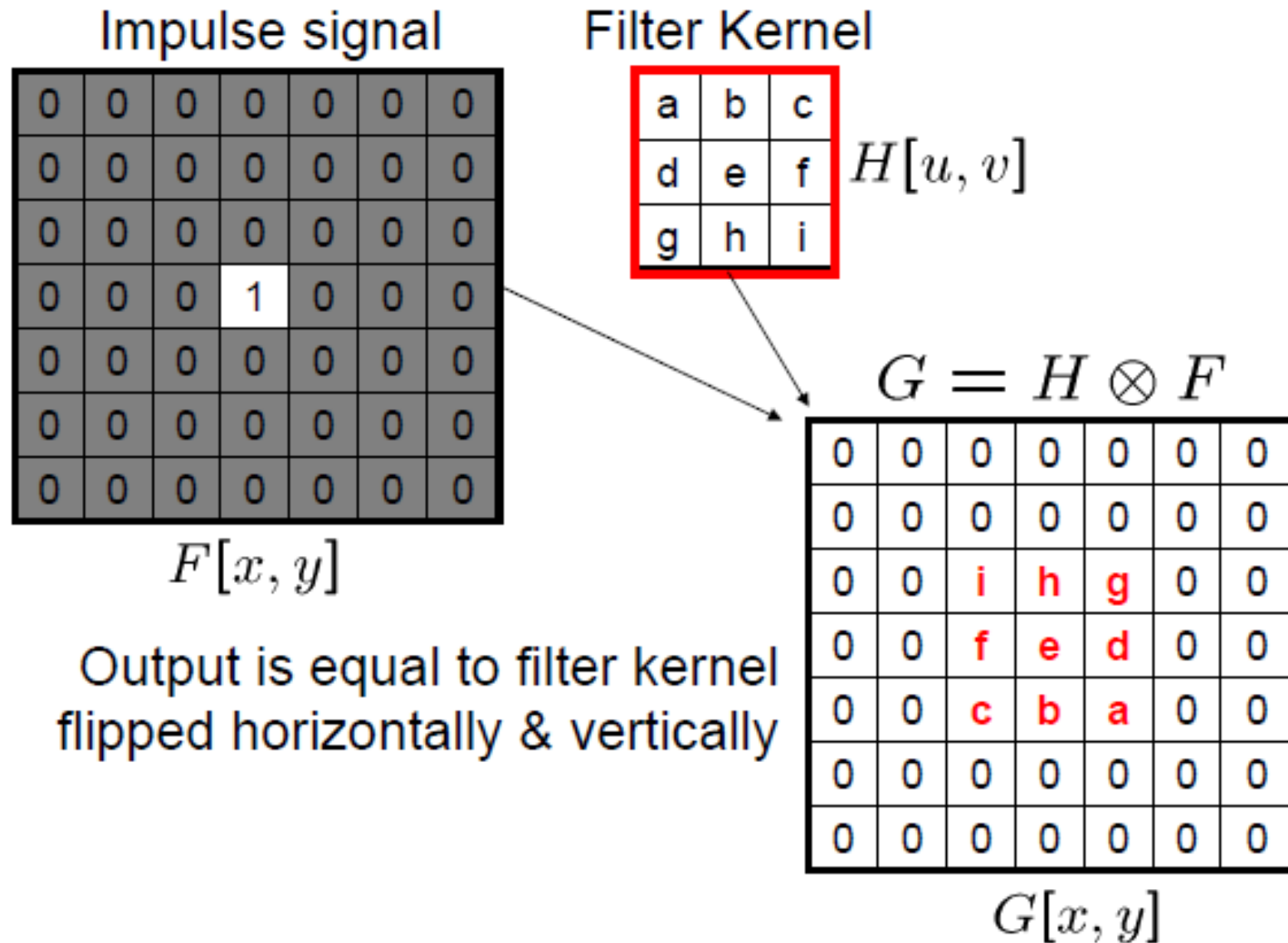


Filtering an impulse signal

What is the result of filtering the impulse signal (image) F with the arbitrary kernel H ?

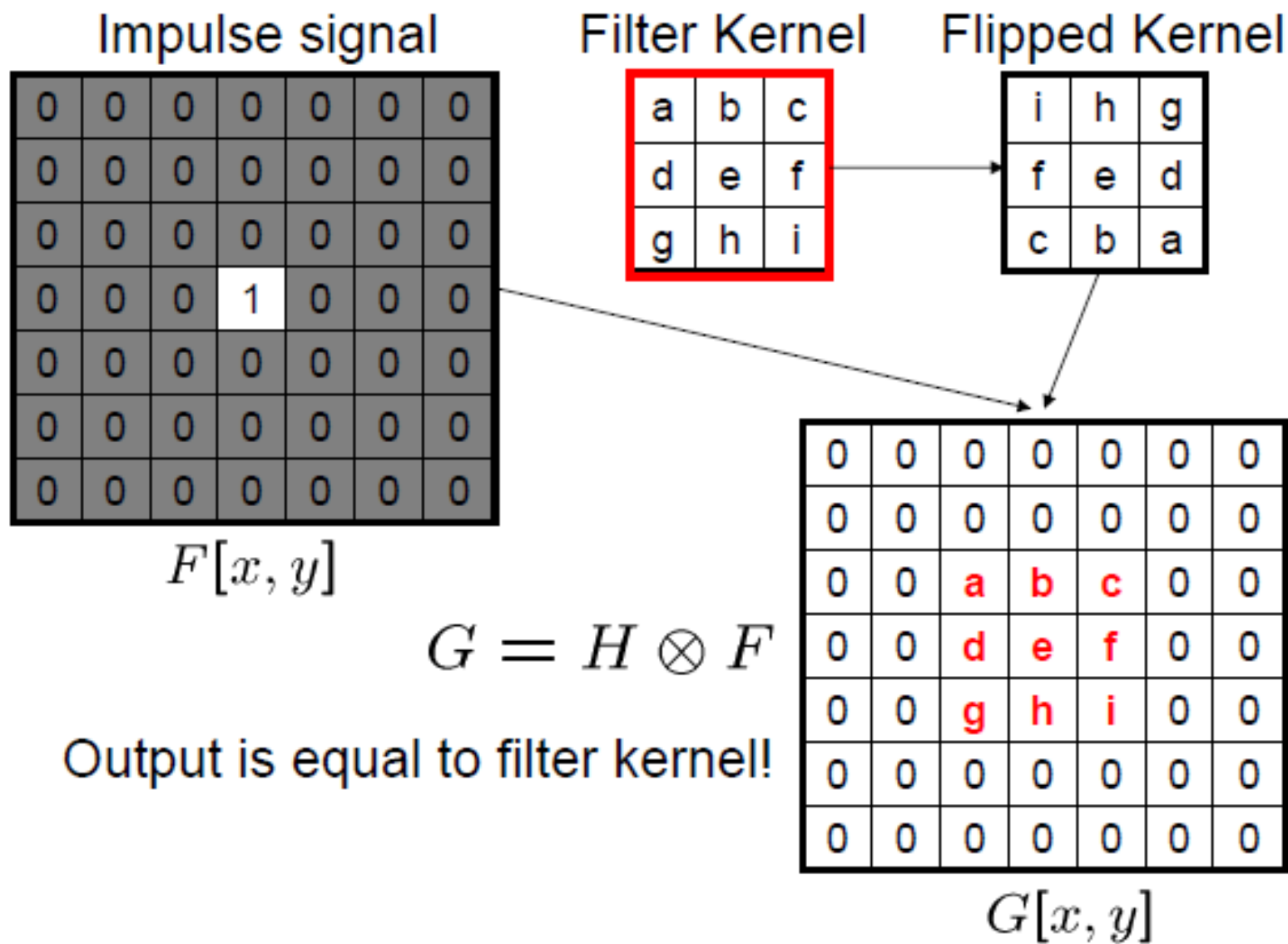


Filtering an impulse



What if we want to get an output that looks exactly like the filter kernel?

Flipping kernels



Predict the outputs using correlation filtering


$$\begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = ?$$


$$\begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 0 & 1 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = ?$$


$$\begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 2 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} - \frac{1}{9} \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} = ?$$

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

?

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Practice with linear filters



Original

$$* \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \right) \quad ?$$

Practice with linear filters



Original

$$* \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \right)$$



Sharpening filter

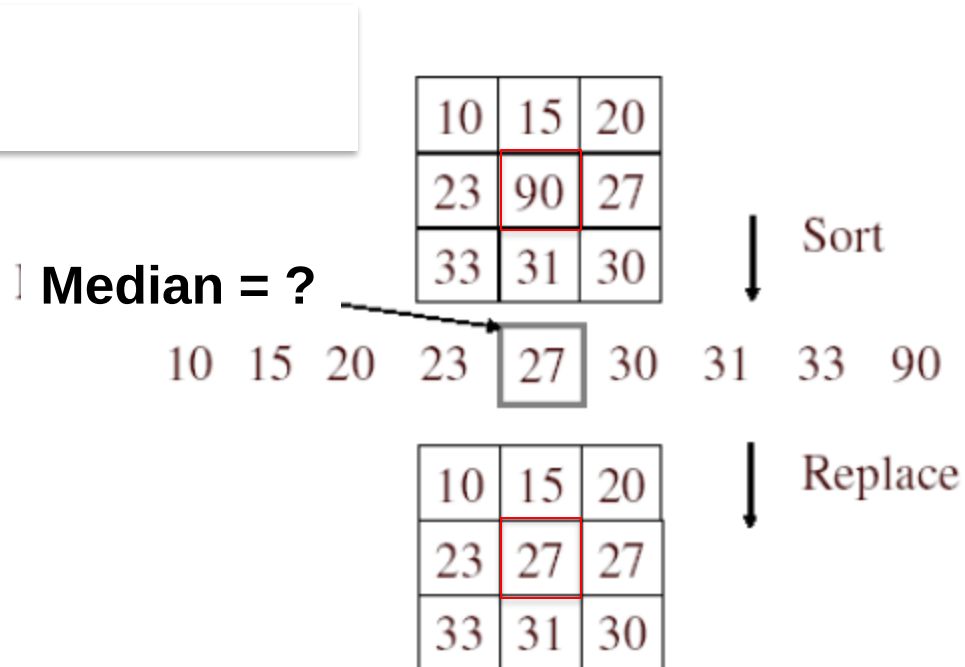
- Accentuates differences with local average

Order-Statistic (Nonlinear) Filtering

- ◆ Output is based on order of gray levels in the masked area.
- ◆ Replacing the value of the center pixel with the value determined by the ranking result.
- ◆ Some simple neighbourhood operations include:
 - **Min:** Set the pixel value to the minimum in the neighbourhood
 - **Max:** Set the pixel value to the maximum in the neighbourhood
 - **Median:** The median value of a set of numbers is the midpoint value in that set

Median filter

A **Median Filter** operates over a window by selecting the median intensity in the window.



- No new pixel values introduced
- Removes spikes: good for impulse, salt & pepper noise
- Non-linear filter

- ♦ Particularly effective when
 - The noise pattern consists of strong impulse noise (salt-and-pepper)

Median filter effect on Salt and Pepper Noise



Salt and pepper noise



Median filtered

Common types of noise

- **Salt and pepper noise:** random occurrences of black and white pixels
- **Impulse noise:** random occurrences of white pixels
- **Gaussian noise:** variations in intensity drawn from a Gaussian normal distribution



Original



Salt and pepper noise



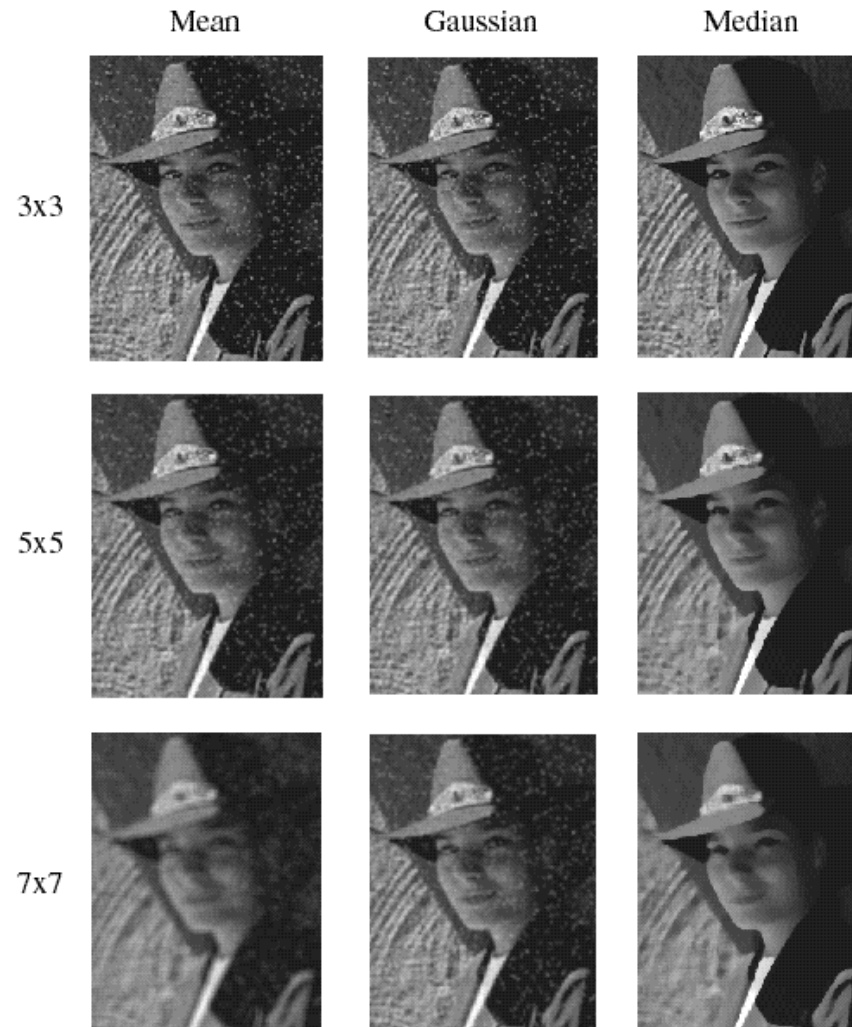
Impulse noise



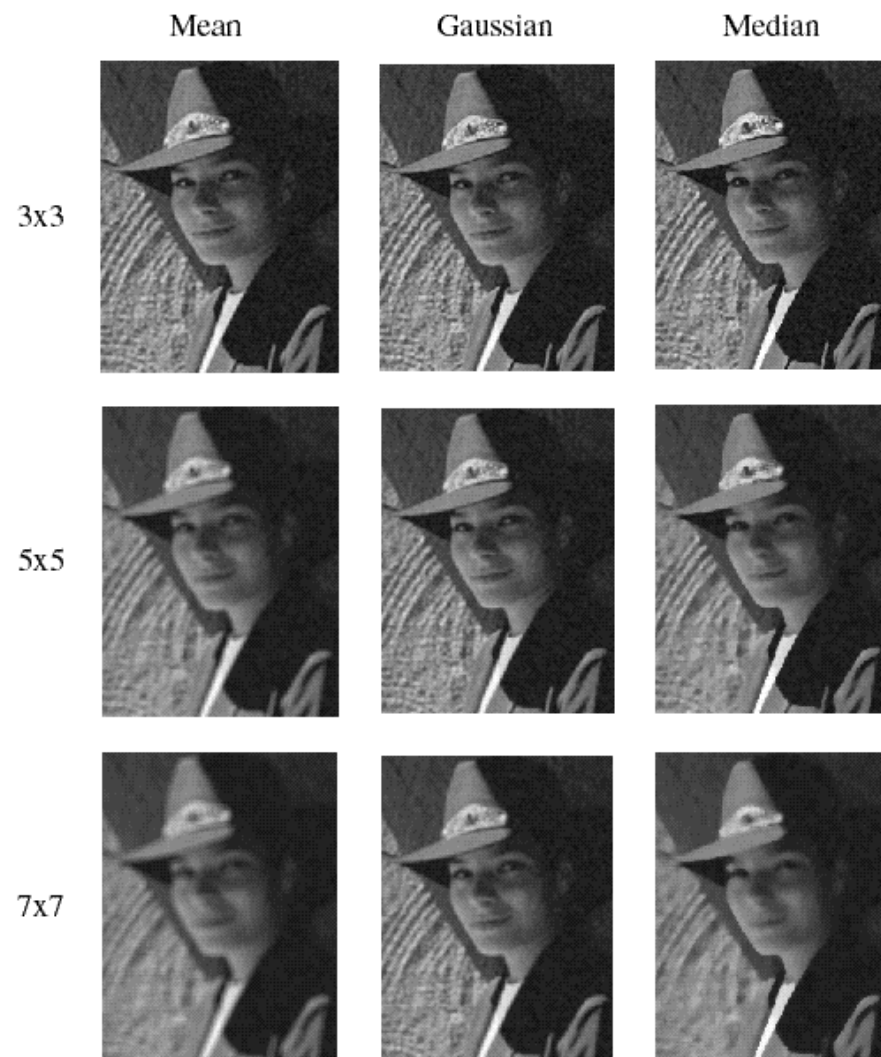
Gaussian noise

Filtering is useful for noise reduction...

Comparison: salt and pepper noise



Comparison: Gaussian noise

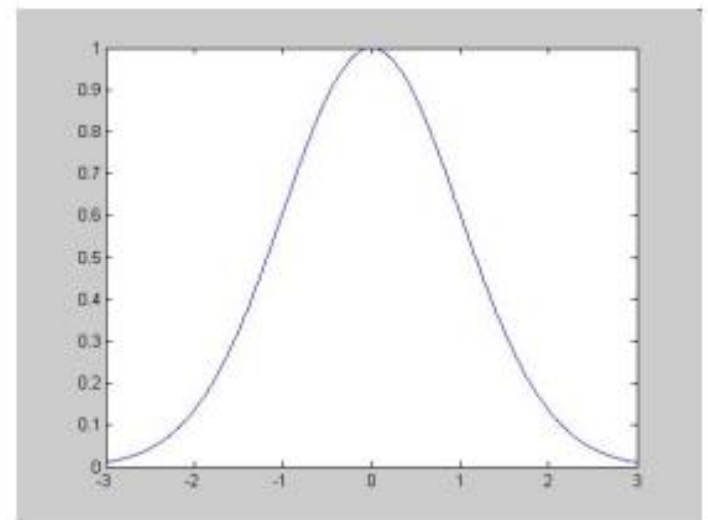


Next time we will looking at sharpening filters
and more on filtering and image
enhancement...>>>

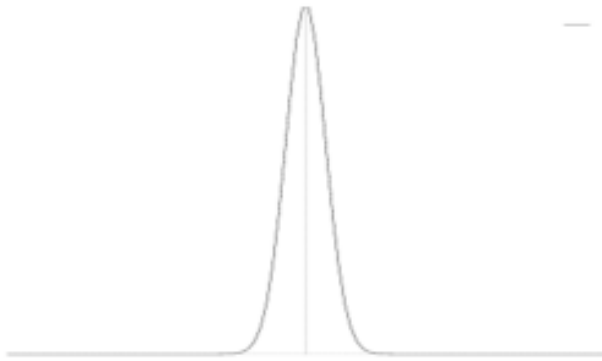
Next slides are extra material for study!!!

Gaussian Noise

$$n(x, y) = e^{\frac{-n^2}{2\sigma^2}}$$

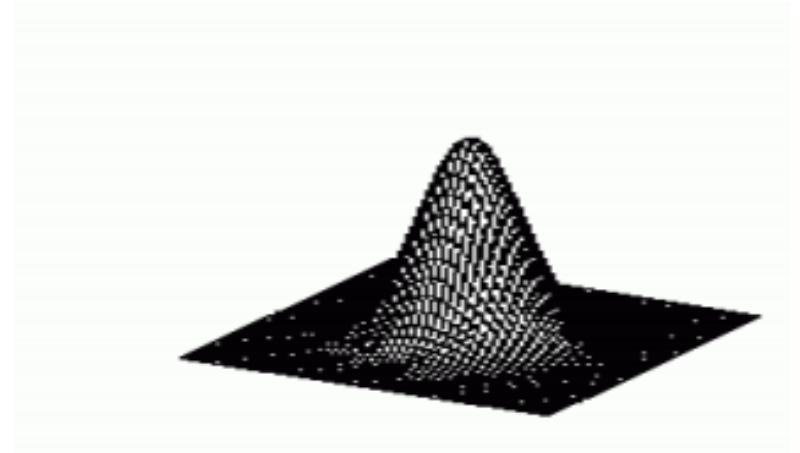


Gaussian filter



$$g(x) = e^{\frac{-x^2}{2\sigma^2}}$$

$$g(x) = [.011 \quad .13 \quad .6 \quad 1 \quad .6 \quad .13 \quad .011]$$



$$g(x, y) = e^{\frac{-(x^2 + y^2)}{2\sigma^2}}$$

$$\sigma = 1$$

Gaussian Filtering

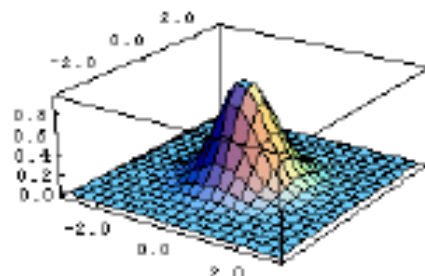
A Gaussian kernel gives less weight to pixels further from the center of the window

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$F[x, y]$

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} H[u, v]$$

Kernel approximates Gaussian function:

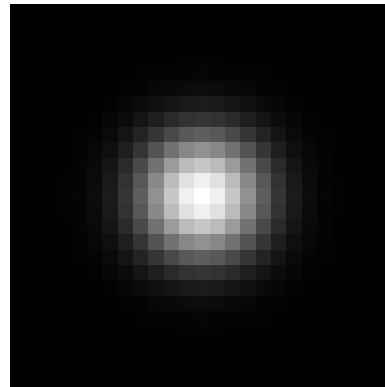
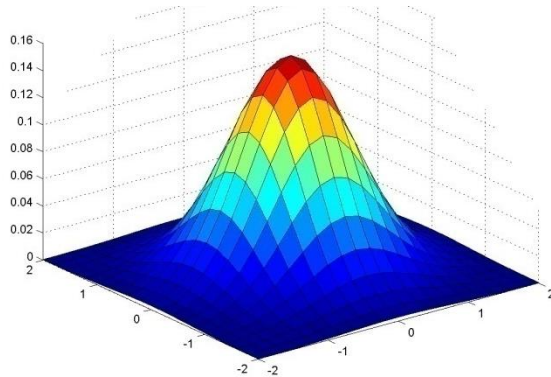


$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$

What happens if you increase σ ?

Important filter: Gaussian

- Spatially-weighted average

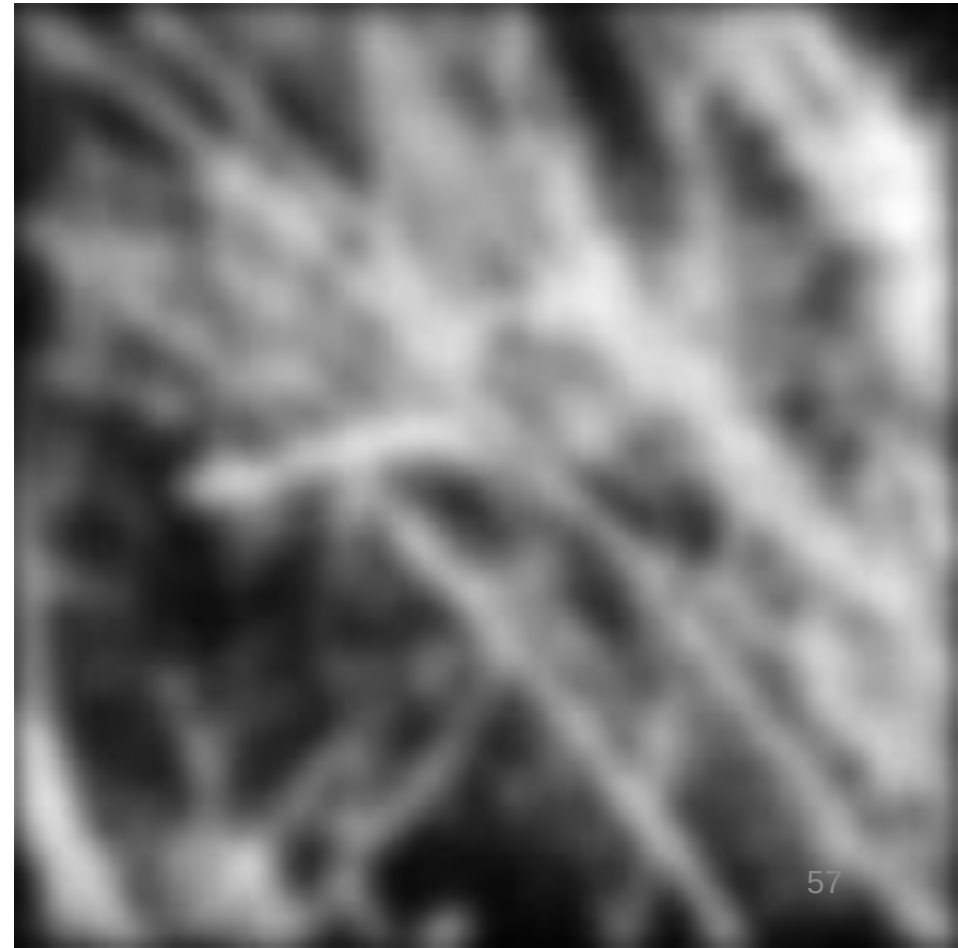


0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

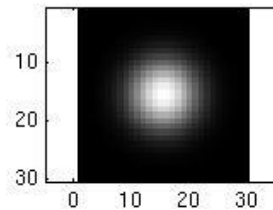
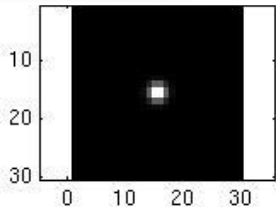
$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Smoothing with Gaussian filter

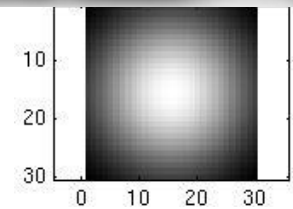


Smoothing with Gaussian filter

Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.

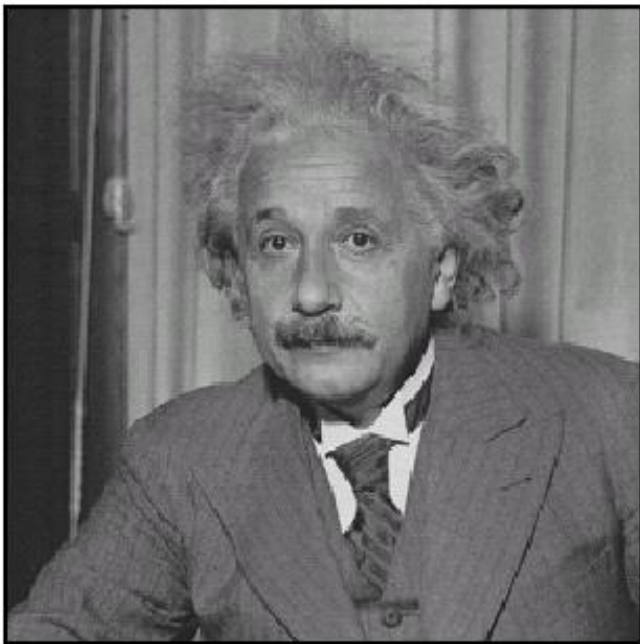


...

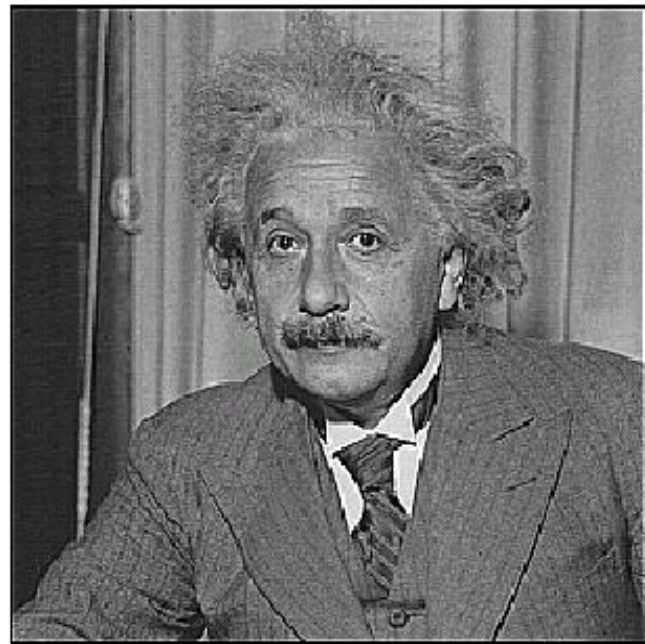


```
for sigma=1:3:10
    h = fspecial('gaussian', fsize, sigma);
    out = imfilter(im, h);
    imshow(out);
    pause;
end
```

Sharpening

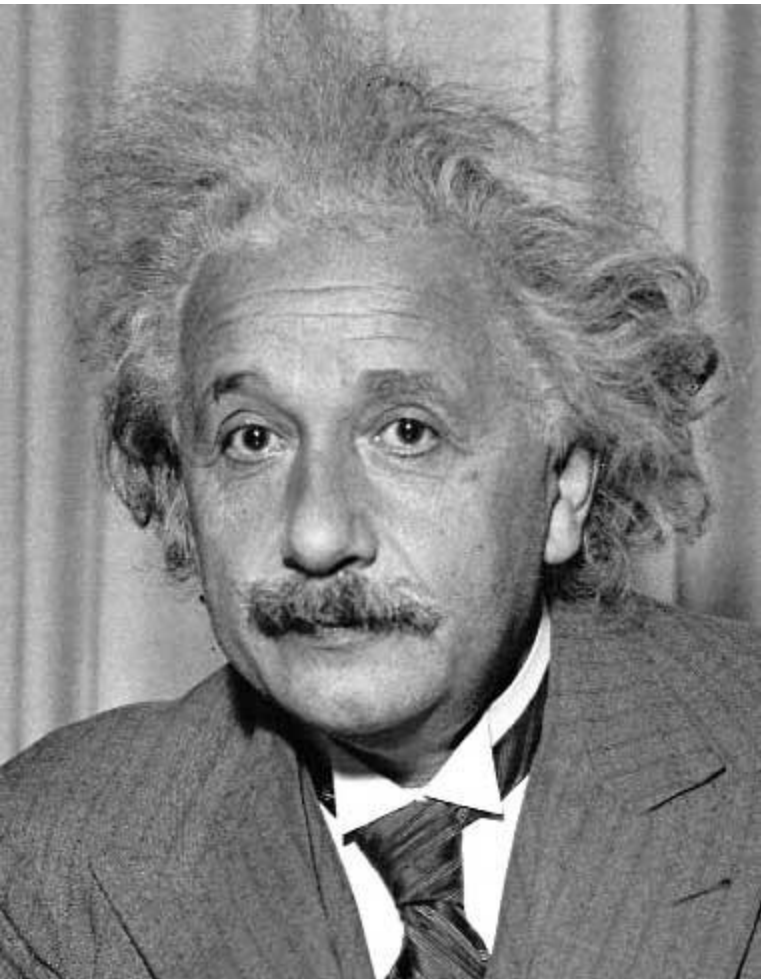


before



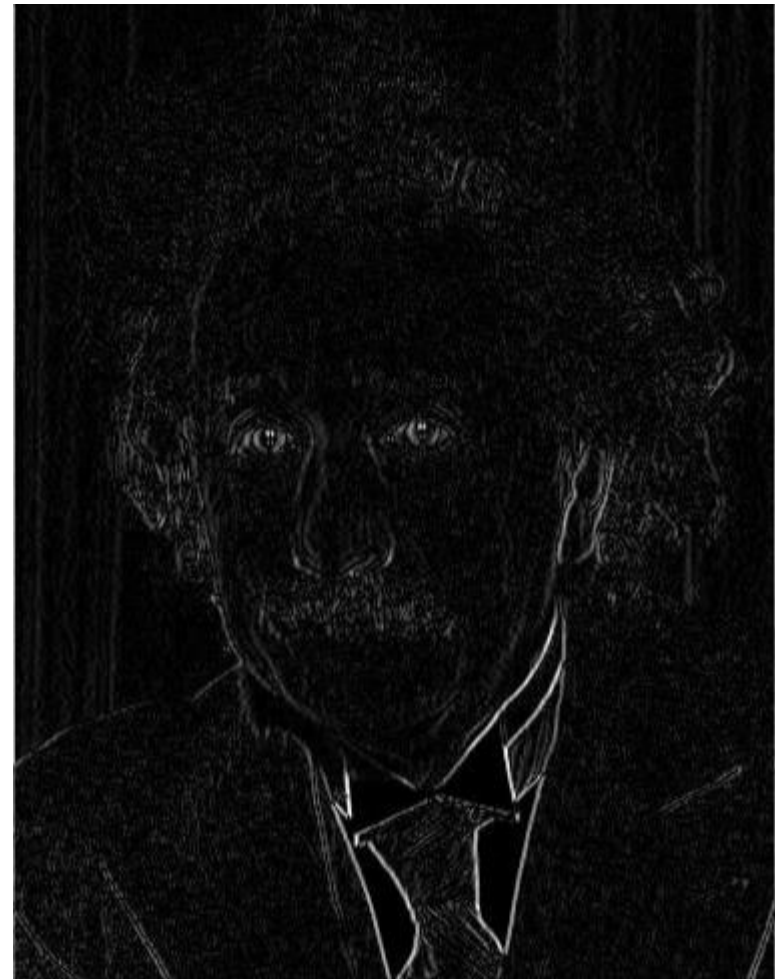
after

Other filters



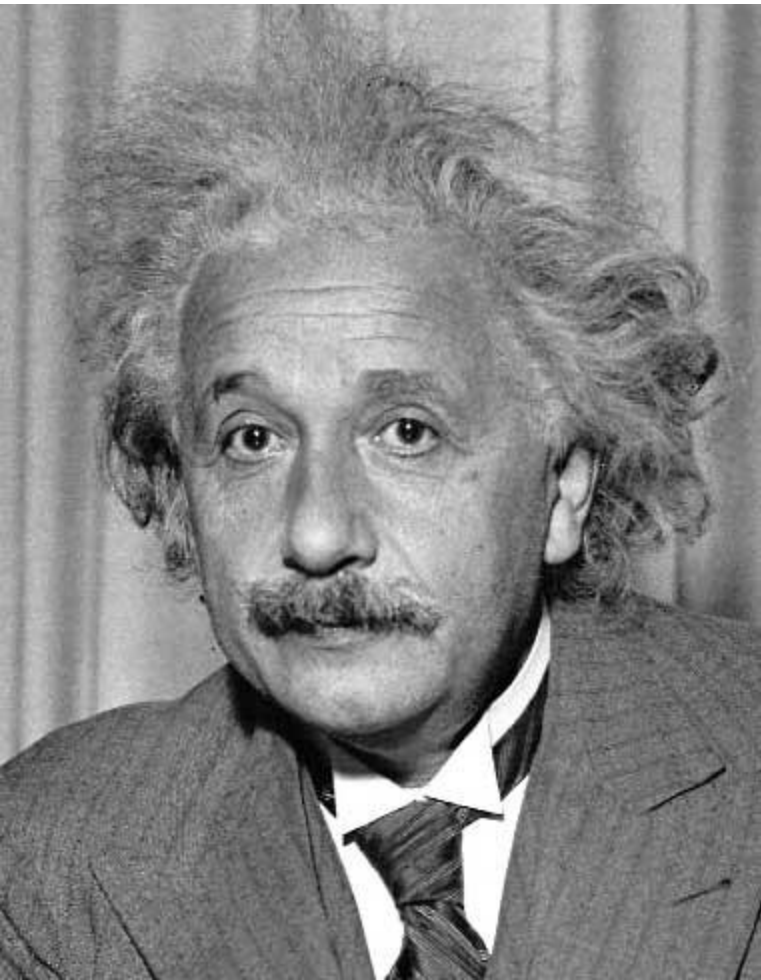
1	0	-1
2	0	-2
1	0	-1

Sobel



Vertical Edge
(absolute value)

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Basic gradient filters

Horizontal Gradient

0	0	0
-1	0	1
0	0	0

or

-1	0	1
----	---	---

Vertical Gradient

0	1	0
0	0	0
0	-1	0

or

-1
0
1

Gaussian filter

- **Remove** “high-frequency” components from the image (low-pass filter)
 - Images become more smooth
- Convolution with self is another Gaussian
 - So can smooth **with small-width** kernel, repeat, and **get same result** as larger-width kernel would have
 - Convoluting two times with Gaussian kernel of width σ is same as convoluting once with kernel of width $\sigma\sqrt{2}$
- *Separable* kernel
 - Factors into product of two 1D Gaussians